

# COLLEGE HELP DESK

Rule-Based FAQ Chatbot in Python

| Field        | Detail                                 |
|--------------|----------------------------------------|
| Title        | Rule-Based FAQ Chatbot                 |
| Scenario     | College Help Desk                      |
| Language     | Python 3.8+                            |
| Level        | Beginner                               |
| Duration     | 1–2 Weeks                              |
| Tools        | Python, VS Code / Google Colab, GitHub |
| FAQs Covered | 20 topics                              |
| Deliverables | Code + Report + Demo Video (optional)  |

**Submitted by:** [Your Name] **Roll No:** [Your Roll No]

**Course:** Introduction to Python / NLP **Date:** [Submission Date]

# 1. Introduction

Chatbots are software programs designed to simulate conversation with users, typically over the Internet or through terminal interfaces. This project implements a **Rule-Based FAQ Chatbot** for a college help desk scenario using core Python 3 features — no machine learning or third-party NLP libraries required.

The chatbot answers questions across **20 FAQ topics** including admissions, fees, examinations, hostel, library, and placements. It detects greetings, farewells, and gratitude, provides topic listings on request, and responds gracefully when a query falls outside its knowledge base.

# 2. Learning Objectives

- Understand the fundamentals of Natural Language Processing (NLP).
- Apply pattern matching and string operations in Python.
- Design a conversation flow using keyword-based intent detection.
- Handle user input/output effectively through the terminal.
- Structure a Python project in a professional, modular fashion.
- Implement session logging using Python file I/O.
- Write and run a self-contained automated test suite.

# 3. System Design

## 3.1 Architecture Overview

The project follows a three-module architecture that separates data, logic, and utilities:

| Module       | Responsibility                                                        |
|--------------|-----------------------------------------------------------------------|
| responses.py | Stores all 20 FAQ entries, keyword lists, and special responses       |
| utils.py     | Input cleaning, keyword matching, display formatting, session logging |
| chatbot.py   | Main conversation loop, intent dispatcher, test suite entry point     |

## 3.2 Intent Dispatch Pipeline

Every user input passes through the following priority chain inside `chatbot.py` → `get_response()`:

| Priority | Intent | Detection Method |
|----------|--------|------------------|
|----------|--------|------------------|

|   |              |                                              |
|---|--------------|----------------------------------------------|
| 1 | Farewell     | Keyword list match (bye, exit, quit ...)     |
| 2 | Greeting     | Keyword list match (hi, hello, hey ...)      |
| 3 | Gratitude    | Keyword list match (thanks, thank you ...)   |
| 4 | Help Request | Exact/partial match (help, topics, menu ...) |
| 5 | FAQ Match    | Scoring: count of FAQ keywords in user input |
| 6 | Fallback     | Random polite message from fallback list     |

## 4. FAQ Topics Covered

The chatbot covers 20 FAQ topics, exceeding the minimum requirement of 15. Each topic has a curated list of trigger keywords and a detailed answer.

| #  | Topic                    | Sample Keywords                         |
|----|--------------------------|-----------------------------------------|
| 1  | Admission                | apply, enroll, register, application    |
| 2  | Fees & Payments          | fee, payment, pay, tuition, due date    |
| 3  | Class Timetable          | timetable, schedule, class timing       |
| 4  | Scholarships             | scholarship, merit, financial aid       |
| 5  | Student ID Card          | id card, identity card, replacement     |
| 6  | Hostel                   | hostel, accommodation, dorm, boarding   |
| 7  | Examinations             | exam, midterm, finals, hall ticket      |
| 8  | Faculty Contact          | faculty, professor, teacher, instructor |
| 9  | Library                  | library, book, borrow, e-library        |
| 10 | Wi-Fi & Internet         | wifi, internet, network, connect        |
| 11 | Certificates & Documents | certificate, bonafide, transcript       |
| 12 | Sports Facilities        | sports, gym, cricket, swimming          |
| 13 | Clubs & Events           | club, society, fest, cultural           |
| 14 | Transport & Bus          | transport, bus, route, bus pass         |
| 15 | Medical & Health         | medical, doctor, clinic, sick           |
| 16 | Portal & Login           | password, login, forgot, reset          |
| 17 | Results & Grades         | result, marks, grade, cgpa              |
| 18 | Attendance               | attendance, absent, leave, shortage     |

|    |                          |                                       |
|----|--------------------------|---------------------------------------|
| 19 | Grievances & Complaints  | complaint, grievance, issue, feedback |
| 20 | Internships & Placements | placement, internship, job, career    |

## 5. Implementation Details

---

### 5.1 Input Cleaning (utils.py)

Raw user input is normalised before any matching takes place. The `clean_input()` function applies three transformations:

- Convert to lowercase — ensures case-insensitive matching.
- Remove punctuation — regex substitution strips non-alphanumeric characters (apostrophes kept).
- Collapse whitespace — multiple spaces reduced to a single space.

### 5.2 Keyword Matching Algorithm (utils.py)

The `find_best_match()` function scores every FAQ entry by counting how many of its keywords appear in the cleaned user input. The FAQ with the highest score is returned. Ties are broken by list order.

This approach is equivalent to a simple bag-of-words similarity measure and handles paraphrased questions effectively — e.g. both *'how do I pay fees'* and *'what is the fee due date'* trigger the Fees & Payments FAQ.

### 5.3 Special Intent Detection (utils.py)

Before FAQ matching, four special intents are checked using dedicated keyword lists:

- **Greeting:** hi, hello, hey, good morning, greetings
- **Farewell:** bye, goodbye, exit, quit, see you
- **Gratitude:** thanks, thank you, thx, appreciated
- **Help Request:** help, topics, menu, list, options

### 5.4 Session Logging (utils.py)

The `SessionLogger` class writes the full conversation to a timestamped text file under the `logs/` directory. Logging can be disabled with the `--no log` flag.

### 5.5 Self-Test Suite (chatbot.py)

Running `python chatbot.py --test` executes 24 automated test cases, one per FAQ topic plus special intents and fallback. Each case verifies that the bot's response contains an expected substring.

## 6. Sample Interaction

---

```
[HelpDesk Bot]: Hello! Welcome to the College Help Desk!
```

```
I can help you with: admissions, fees, timetable, hostel, exams...
```

[You ]: how do i apply for admission?

[HelpDesk Bot]: To apply for admission:

1. Visit our website and click 'Apply Now'.

2. Fill the form and upload documents.

3. Pay the application fee (Rs. 500).

Reviews completed in 5-7 working days.

[You ]: what scholarships are available?

[HelpDesk Bot]: Available Scholarships:

1. Merit Scholarship - Top 5%, 50% tuition waiver

2. Need-Based Aid - Income < Rs. 3L/yr, up to 100% waiver

3. Sports/Cultural - State-level achievers, 25% waiver

Apply by July 15 every year.

[You ]: bye

[HelpDesk Bot]: Thank you for using the College Help Desk! Have a great day!

## 7. Python Concepts Demonstrated

| Concept                  | Location     | Description                                 |
|--------------------------|--------------|---------------------------------------------|
| String methods           | utils.py     | .lower(), .strip(), .split(), 'in' operator |
| Regular expressions (re) | utils.py     | re.sub() for input sanitisation             |
| Functions & modularity   | all modules  | Single-responsibility functions             |
| Lists & dictionaries     | responses.py | FAQ data as list of dicts                   |
| Loops & conditionals     | chatbot.py   | while loop, if-elif-else dispatch           |
| File I/O                 | utils.py     | SessionLogger writes .txt logs              |
| Exception handling       | chatbot.py   | try/except for KeyboardInterrupt/EOF        |
| Module imports           | chatbot.py   | Imports from responses & utils              |
| Random module            | utils.py     | random.choice() for fallback variety        |
| Command-line args        | chatbot.py   | sys.argv parsing for --test/--nolog         |
| Type hints               | utils.py     | str, list, bool, dict   None                |

## 8. Testing

The self-test suite in `chatbot.py` contains 24 test cases covering all 20 FAQ topics, all special intents, and the fallback handler. Tests assert that the response string contains an expected keyword fragment.

| Test Query                   | Expected Fragment | Status |
|------------------------------|-------------------|--------|
| hello                        | Welcome           | PASS   |
| how do i apply for admission | Admission         | PASS   |
| what are the fees            | Fee Payment       | PASS   |
| scholarship for merit        | Merit Scholarship | PASS   |
| hostel application           | Hostel            | PASS   |
| when are exams               | Examination       | PASS   |
| check my result              | Results           | PASS   |
| attendance shortage          | 75%               | PASS   |
| campus placement             | Placement         | PASS   |
| xyzabc123 (fallback)         | sorry             | PASS   |
| bye                          | Thank you         | PASS   |
| ... (14 more cases)          | ...               | PASS   |

## 9. Conclusion

This project successfully demonstrates the implementation of a rule-based FAQ chatbot using Python 3. The chatbot covers 20 frequently asked questions for a college help desk, handles unrecognised inputs gracefully, logs sessions to disk, and includes a 24-case automated test suite.

The modular architecture — separating data (`responses.py`), logic (`utils.py`), and the main loop (`chatbot.py`) — makes the codebase easy to extend. Adding a new FAQ requires only one new dictionary entry in `responses.py`.

Possible future enhancements include: integration with the `re` module for regex-based matching, a GUI front-end using Tkinter, spell-correction using `difflib`, or upgrading to a machine-learning model (e.g. TF-IDF + cosine similarity) for better generalisation.